



Chair of
Information Theory
and Data Analytics

RWTHAACHEN
UNIVERSITY

RESOURCE-EFFICIENT INFERENCE IN LARGE GENERATIVE MODELS

PATRICK TAZO KUETE

A seminar submitted to the
Faculty of Electrical Engineering and Information Technology,
RWTH Aachen University,
supervised by Hazem el Hadj Khalifa, M.Sc.

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS
RWTH AACHEN UNIVERSITY

Abstract

Large generative models provide strong capabilities in natural-language processing, code generation, and reasoning, but their inference requires substantial computation, memory capacity, memory bandwidth, and energy. These requirements present a particular challenge for deployment on resource-constrained edge devices.

This paper surveys resource-efficient inference in decoder-based large language models. It first outlines the relevant components of the Transformer architecture and examines the two principal stages of inference: the parallel prefill phase and the sequential autoregressive decode phase. This analysis identifies the main resource bottlenecks, including the computational and memory cost of attention, repeated model-weight transfers, and the growth of the Key–Value cache.

Based on these bottlenecks, the paper reviews optimisation methods at several levels of the inference stack. These include model compression through quantisation, pruning, and knowledge distillation; optimised attention computation and Key–Value cache management; decoding and scheduling methods such as speculative decoding and continuous batching; and system-level runtimes including `llama.cpp`, MLC-LLM, and vLLM. The methods are compared according to the resources they reduce, their accuracy and performance trade-offs, and their suitability for different deployment environments.

The central finding is that resource-efficient inference cannot be achieved through a single universally optimal method. Effective deployment requires identifying the dominant bottleneck of the target platform and combining compatible model-level, algorithmic, decoding, and system-level optimisations accordingly.

Contents

List of Abbreviations	ix
1 Introduction	1
2 Preliminaries and Terminology	3
2.1 Transformer Architecture	3
2.2 Inference in LLMs	4
2.3 Terminology and Key Metrics	5
3 Towards Efficient Inference: Techniques and Solutions	7
3.1 Model Compression	9
3.1.1 Quantization	9
3.1.2 Pruning	10
3.1.3 Knowledge Distillation	11
3.2 Inference-Level Optimisations	12
3.2.1 Optimised Attention Computation	12
3.2.2 KV Cache Optimisation	13
3.3 Decoding Acceleration and Request Scheduling	15
3.3.1 Speculative Decoding	15
3.3.2 Continuous Batching	15
3.4 System-Level Inference Runtimes	16
3.4.1 llama.cpp	16
3.4.2 MLC-LLM	17
3.4.3 vLLM	17
3.5 Comparative Discussion and Edge-Deployment Guidelines	18
3.5.1 The Efficiency–Accuracy Trade-off	20
3.5.2 Memory as a Hard Edge-Deployment Constraint	20
3.5.3 Technique Combination and Deployment Selection	21
4 Conclusion	23
Bibliography	25

List of Figures

List of Abbreviations

FFN Feed-Forward Network 11

FLOPs Floating Point Operations per Second 5, 10

KD Knowledge Distillation 11

KV Key-Value 1, 6, 12–14, 17

LLM Large Language Model 7

MLC-LLM Machine Learning Compilation for Large Language Models 2

QAT Quantisation-Aware Training 9

TPOT Time per Output Token 6

TTFT Time to First Token 6

1 Introduction

Large generative models have significantly advanced the capabilities of Artificial Intelligence (AI) systems in areas such as natural-language processing, code generation, question answering, and reasoning. Much of this progress is based on the Transformer architecture, whose attention mechanism enables models to construct context-aware representations of input sequences [17]. Increasing the number of model parameters and the amount of training data has further improved the range of tasks that these models can perform, as demonstrated by model families such as GPT and LLaMA [2, 16].

However, the capabilities of these models are accompanied by considerable inference requirements. Depending on the model size, sequence length, numerical precision, and deployment hardware, generating an output typically requires substantial computation, memory capacity, memory bandwidth, and energy. These requirements are especially restrictive on edge devices such as smartphones, embedded systems, and other platforms that operate with limited memory, processing capacity, and power budgets. Consequently, deploying large generative models outside specialised data centres often requires optimisations at the model, algorithmic, and system levels [1, 22].

A clear understanding of the inference process is necessary before these optimisation methods can be evaluated. In decoder-based language models, inference consists of an initial prefill phase followed by sequential autoregressive decoding. The two phases exhibit different resource characteristics: prefill processes the prompt largely in parallel and can be compute-intensive, whereas decoding repeatedly accesses the model parameters and the growing Key-Value (KV) cache for every generated token. The dominant bottleneck may therefore vary between computation, memory capacity, memory bandwidth, latency, and energy depending on the workload and target platform [21, 22].

Against this background, this report addresses the following research questions:

- Which operations and data structures constitute the main resource bottlenecks during Transformer inference?
- How do model compression, inference-level optimisation, decoding acceleration, and system-level runtimes address these bottlenecks?
- What trade-offs determine whether a particular optimisation is suitable for resource-

constrained edge devices or server-based deployment?

To answer these questions, the report first introduces the relevant components of the Transformer architecture and examines the prefill and decode phases of inference. It then surveys model-compression techniques, including quantisation, pruning, and knowledge distillation; inference-level methods for attention and KV-cache optimisation; decoding and scheduling techniques such as speculative decoding and continuous batching; and system-level runtimes including `llama.cpp`, Machine Learning Compilation for Large Language Models (MLC-LLM), and vLLM. The methods are considered according to the bottlenecks they address, their resource–accuracy trade-offs, and their suitability for different deployment environments.

2 Preliminaries and Terminology

2.1 Transformer Architecture

The Transformer model has brought considerable advancements to the field of Natural Language Processing (NLP). Unlike sequential traditional models such as RNNs (Recurrent Neural Networks), the Transformer architecture uses an innovative attention mechanism to keep track of long-range dependencies between tokens derived from the input sequence. This is enabled by a clever mechanism, which enriches the token representations (Embeddings) by taking into consideration the meaning of the other surrounding tokens: Attention [17]

Prior to enriching the input tokens with contextual meaning, the latter is first mapped to a vector in a d_k dimensional space where the meaning of the word is encoded in a first instance. This is known as Embedding, and the resulting vectors are often referred to as input embeddings. Following this, knowledge about the position of each token is also needed and this information is represented using positional encodings, usually generated by sinusoidal functions. These positional encodings are then added to the input embeddings, as Transformers do not inherently have knowledge of the position of each token.

The Attention mechanism for a given set of input tokens starts by deriving three vectors for each token, namely: the Query, Key and Value vectors by making a linear projection with learned projection matrices. A dot product between all the query and key vectors of each input embedding is then computed to find out which tokens are compatible (in the formula below, this is expressed as a matrix multiplication of the Query and Key matrices). In order to prevent the dot products to grow too large and push the softmax function into areas where the gradients are extremely small, the dot products are normalised by $\sqrt{d_k}$. The values obtained after the softmax are then used as weights which are multiplied with each value vector and added

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where, Q, K, and V represent the query, key, and value matrices, which are a compact representation for the different query, key and value vectors of all input tokens, and d_k is the dimension of the key vectors.

as previously mentioned, builds on this self-attention mechanism, by carrying out the same operation in parallel with different learned weights and concatenating the output of each individual head. In order to capture other possible relationships between the input tokens which would not necessarily be captured by a single attention head, the Multi-Head Self-Attention is introduced. This extends the Self-Attention mechanism by computing different Attention heads in parallel for different sets of learned projection matrices of the Query, Key and Value and then concatenating the results. This can be represented as follows:

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_h)W^O$$

where $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$ stands for the i -th attention head output, and W^O is the output projection matrix. [1]

As can be seen from the attention formula above, the matrix multiplication between Q and K^T , each of dimension $(n \times d_k)$ and $(d_k \times n)$ respectively, brings about a time complexity of $O(n^2)$, where n denotes the input sequence length. The subsequent multiplication with the value matrix accounts for a complexity of $O(d)$. Putting these two together, we see that the time complexity for the Attention mechanism amounts to $O(n^2d)$. The time complexity of the Attention mechanism therefore grows quadratically with the input sequence, which could bring about computational difficulties for very long input sequences. Other than time complexity, it is worth taking a look at memory complexity as well. This amounts to $O(n^2)$ as result of the $(n \times n)$ attention matrix (QK^T) needing to be stored for further computation.

2.2 Inference in LLMs

Inference in decoder-based Large Language Models (LLMs) is autoregressive: the model generates one token at a time, and each generated token becomes part of the input used to predict the next token. The inference process is commonly divided into two stages: the *prefill phase* and the *decode phase* [21, 22].

During the prefill phase, the complete input prompt is tokenised, embedded, and processed through all Transformer layers in a single forward pass. Since all prompt tokens are known in advance, their representations can be processed largely in parallel. Within every self-attention layer, query, key, and value vectors are computed for the prompt tokens. The resulting key and value vectors are stored in the Key-Value (KV) cache for subsequent decoding steps. Consequently, constructing the KV cache is a result of the prefill computation rather than its sole purpose. For a prompt containing n tokens, standard

self-attention computes an $n \times n$ attention matrix and therefore has a computational complexity of approximately $\mathcal{O}(n^2d)$, where d denotes the hidden dimension. The prefill phase can therefore become computationally expensive for long input sequences [17, 21].

After the prompt has been processed, the decode phase generates the output autoregressively. At each decoding step, the previously generated token is embedded and passed through the Transformer layers. A new query, key, and value vector is computed in each attention layer. The new query attends to the keys of all previously processed prompt and output tokens stored in the KV cache, while the newly computed key and value vectors are appended to that cache. The resulting hidden representation is passed through the language-model head to obtain a probability distribution over the vocabulary, from which the next token is selected.

Unlike prefill, decoding is inherently sequential because token $t + 1$ cannot be computed before token t has been generated. Although the KV cache avoids recomputing the keys and values of previous tokens, its memory consumption grows linearly with the total sequence length. Furthermore, the model parameters and cached values must be accessed for every generated token. As a result, the decode phase is frequently constrained by memory capacity and memory bandwidth rather than by arithmetic throughput [21, 22]. This distinction between the parallel, compute-intensive prefill phase and the sequential, often bandwidth-bound decode phase is important because the two stages require different optimisation strategies.

2.3 Terminology and Key Metrics

The following terms are consistently throughout this report.

Computation: Computation refers to the amount of arithmetic processing required to perform inference. It is typically measured by the number of Floating Point Operations per Second (FLOPs) or integer operations needed to generate an output.

Memory: Memory refers to the amount of storage required to hold the data needed during inference, including model parameters, intermediate activations, and the $|kv|$ cache, measured in Bytes.

Energy: Total amount of electrical energy consumed during the inference process measured in Joules.

Bandwidth: maximum amount of data that can be transferred between memory and the processor per unit time. In our case, this could for example be how fast the parameters of the LLM are loaded into the GPU.

KV cache: The stored key and value tensors for all past tokens, maintained to avoid recomputation at each decode step.

Quantization: Representing weights and/or activations at reduced numerical precision to reduce memory footprint and, on hardware with native low-precision arithmetic, increase throughput.

Pruning: Removing parameters from a trained model. *Unstructured* pruning zeros individual weights; *structured* pruning removes entire rows, columns, heads, or layers to yield a dense, smaller model directly executable on standard hardware.

Knowledge Distillation: Training a compact *student* model to match the output distribution of a larger *teacher*, transferring capability at a fraction of the inference cost.

Throughput: Tokens (or requests) completed per second across all concurrent users measured in tokens per second.

Latency: Elapsed time from request arrival to final-token delivery. Time to First Token (TTFT) (time to first token) governs perceived responsiveness; Time per Output Token (TPOT) (time per output token) governs generation speed.

Edge device: Any computing platform with constrained memory, power, and compute operating outside a data centre: smartphones or embedded controllers.

3 Towards Efficient Inference: Techniques and Solutions

Building upon the previous chapter, this chapter systematically surveys the techniques that have been developed to reduce the resource demands of Large Language Model (LLM) inference. For each technique the discussion covers its core mechanism, the bottleneck it addresses, its accuracy–efficiency trade-off, and its practical implications for edge deployment. In table 3.1 the principal inference bottlenecks identified in Chapter 2 are mapped to the optimisation techniques discussed in this chapter. Since several methods address more than one resource constraint, the categories are not mutually exclusive.

Inference bottleneck	Relevant techniques	Primary effect
Weight memory and memory bandwidth	Weight quantisation(section 3.1.1), pruning section 3.1.2, knowledge distillation ??	Reduces the model size and the amount of weight data transferred from memory during inference.
KV-cache memory	KV-cache quantisation ??, PagedAttention ??, knowledge distillation ??	Reduces the size of the KV cache or improves how its memory is allocated and managed.
Attention computation and memory traffic	FlashAttention ??, pruning ??	Reduces attention-related memory accesses, intermediate storage, or arithmetic operations.
Sequential autoregressive decoding	Speculative decoding ??	Allows multiple candidate tokens to be verified in one target-model forward pass, reducing the number of sequential decoding steps.
Low hardware utilisation	Continuous batching ??, PagedAttention ??	Improves accelerator utilisation by dynamically grouping requests and using memory more efficiently.
Hardware portability and kernel efficiency	llama.cpp refsubsubsection:llama-cpp, MLC-LLM refsubsubsection:mlc-llm, vLLM refsubsubsection:vllm	Provides hardware-specific kernels, runtime optimisations, and memory-management strategies for different deployment platforms.

Table 3.1: Mapping of major LLM inference bottlenecks to the efficiency techniques that address them.

3.1 Model Compression

Model compression techniques reduce the size of the model itself shrinking weight memory and, consequently, the bandwidth required to load those weights during each decode step. They are pre-deployment operations and impose a one-time cost in compute or data for calibration.

3.1.1 Quantization

Quantization is a model-compression strategy that lowers the numerical precision of model parameters and/or intermediate values. Instead of storing and computing with high-precision formats such as FP32, FP16 or BF16, quantized models use lower-bit formats such as INT8, INT4, or even lower representations. This is especially useful for edge deployment, where memory capacity, bandwidth, energy consumption, and computational resources are limited. However, quantizing large language models is more difficult than quantizing many conventional neural networks because Transformer architectures contain attention layers, high-dimensional hidden states, and activation distributions with large dynamic ranges. These properties make LLMs sensitive to numerical approximation, so aggressive quantization can reduce model quality if it is not carefully controlled. [20]

Quantization methods can be classified according to when quantization is introduced and which tensors are quantized. Post Training Quantization(PTQ) modifies an already trained model, and requires retraining which can be very costly, whereas Quantisation-Aware Training (QAT) incorporates simulated quantization during training or fine-tuning. Independently, a method may quantize only the weights or both weights and activations [22].

Weight-only quantization

In weight-only quantization the precision of the model’s stored parameters (weights) are reduced while usually keeping activations in higher precision. This mainly lowers memory usage and can improve inference efficiency. Examples include LLM.int8(), which uses vectorwise quantization and mixed-precision decomposition enabling Int8 matrix multiplication for feed-forward and attention projection layers in transformers. This has been shown to have minimal precision loss and maintain performance for models with upto 175B parameters [4]. Another method known as GPTQ takes compression even further by reducing the bitwidth down to 3 or 4 bits per weight for models with up to 175B parameters without significant accuracy loss. This method uses approximated second-order information which allows it to identify sensitive weights and compensate for quantization errors by adjusting the remaining weights [7].

Weight-activation (co-)quantization

Weight-activation (co-)quantization goes further by reducing the precision of both weights and intermediate activations. This can provide additional memory and computational savings, but it is harder because activations in LLMs often contain large outliers and irregular value ranges. Methods such as ZeroQuant and SmoothQuant address this by using more refined scaling or grouping strategies. In particular, SmoothQuant shifts part of the quantization difficulty from activations to weights through mathematically equivalent scaling transformations, enabling efficient 8-bit weight-and-activation quantization with limited accuracy loss. [20]

3.1.2 Pruning

Pruning removes parameters from the model that contribute least to its predictions in accordance with some pruning criterion such as minimising the reconstruction loss of the model [22]. This in turn brings about a reduction in both weight memory and FLOPs count. The two principal strategies: *unstructured* and *structured* pruning differ in *what* they remove and, consequently, in the hardware speedup they deliver. *unstructured* pruning sets individual weights to zero while preserving the original tensor dimensions. Consequently, the reduction in the number of non-zero parameters does not automatically produce a proportional reduction in inference latency. Practical acceleration depends on sparse storage formats, sparse-aware kernels, and compatible hardware support. *Structured* pruning instead removes complete regular structures, such as rows, columns, attention heads, or hidden dimensions, thereby producing smaller dense operations that can generally be executed using conventional dense linear-algebra kernels [5, 18].

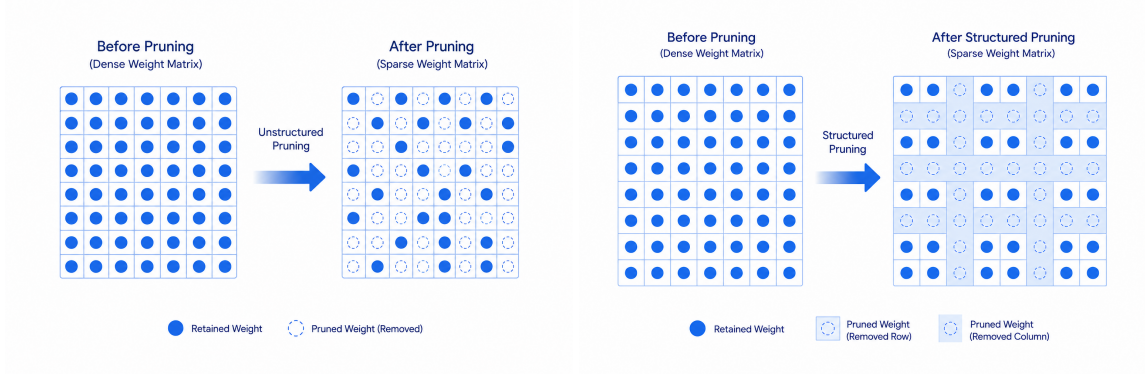
Unstructured pruning

Unstructured pruning removes or sets individual scalar weights to zero, producing an irregular sparsity pattern while retaining the original tensor dimensions. SparseGPT is a one-shot post-training example of an unstructured pruning method designed for large-scale generative language models. The method was evaluated on models including OPT-175B and BLOOM-176B and was able to reach 50–60% unstructured sparsity without retraining. At 60% sparsity, the authors report only a small increase in language-model perplexity, while the pruning procedure required less than 4.5 hours for the largest evaluated models [6]. These results show that a large proportion of individual weights can be removed without substantially degrading predictive quality. However, the reported sparsity does not by itself imply an equivalent reduction in inference latency, since practical acceleration depends on sparse storage formats, sparse-aware kernels, and suitable hardware support.

Structured pruning

Structured pruning removes entire, regular sub-structures such as rows or columns of weight matrices, attention heads, Feed-Forward Network (FFN) intermediate dimensions, or whole Transformer layers yielding a strictly smaller *dense* matrix that requires no sparse encoding and executes efficiently on any standard hardware. LLM-Pruner removes coupled structures, such as groups of channels, using gradient-based importance estimates. On Vicuna-7B, pruning approximately 20% of the parameters retained about 92% of the original model’s average zero-shot performance across the evaluated classification benchmarks [14]. The method also supports performance recovery through LoRA-based post-training using 50 000 training samples, which the authors report can be completed in approximately three hours. At substantially higher pruning ratios, however, the performance degradation becomes considerably larger, indicating that structured pruning offers direct reductions in dense model size and computation at the cost of a stronger compression–accuracy trade-off.

The reported results should be interpreted with care because the two methods use different pruning structures and evaluation metrics. SparseGPT primarily reports the effect of unstructured sparsity on language-model perplexity, whereas LLM-Pruner evaluates structured parameter removal using average downstream-task performance. Consequently, the reported percentages are not directly comparable.



3.1.3 Knowledge Distillation

Knowledge Distillation (KD) is a model compression technique in which a smaller student model is trained to imitate the behavior of a larger teacher model. In the context of large language models, KD is used to transfer knowledge and capabilities from powerful LLMs into smaller language models, reducing memory and computational requirements while preserving as much performance as possible. KD methods for LLMs are commonly divided into white-box and black-box approaches. [22]

White-box knowledge distillation. White-box knowledge distillation assumes access to the internal output distribution or intermediate representations of the teacher model. A representative example is MiniLLM, which distils an accessible large language model into a smaller generative model using the teacher’s token-level probability distribution. Unlike conventional knowledge distillation, which commonly minimises the forward Kullback–Leibler divergence, MiniLLM employs the reverse Kullback–Leibler divergence. This objective discourages the student from assigning probability mass to low-probability regions of the teacher distribution and was found to improve response quality, calibration, and long-text generation. The method was evaluated across teacher and student models ranging from 120 million to 13 billion parameters [9].

Black-box knowledge distillation. Black-box knowledge distillation is used when the teacher’s parameters, hidden states, and output logits are unavailable. In this setting, the teacher is accessed only through its generated responses. Distilling Step-by-Step is a representative method in which a large language model is prompted to generate both task labels and natural-language rationales for training examples. These outputs are then used as additional supervision in a multi-task objective for a smaller student model. The rationales provide intermediate task knowledge that is not contained in the final label alone, allowing the student to learn with less training data. In the reported experiments, a 770-million-parameter T5 student outperformed a few-shot-prompted 540-billion-parameter PaLM model on one benchmark while using only 80% of the available training data [10].

The principal distinction is therefore the information available from the teacher. White-box methods can directly align the student’s probability distribution or internal representations with those of the teacher, but require access to the teacher model. Black-box methods are applicable to proprietary models exposed through an API, although the student can learn only from the text, labels, or rationales generated by the teacher.

3.2 Inference-Level Optimisations

These techniques modify the attention mechanism or the KV cache management strategy without altering the model weights. They can be applied at inference time, often transparently to the model.

3.2.1 Optimised Attention Computation

Standard self-attention requires the computation of an $n \times n$ matrix of attention scores, where n is the sequence length. Its arithmetic complexity therefore grows quadratically

with the sequence length. A conventional implementation also materialises the intermediate score and probability matrices in GPU memory, resulting in considerable memory traffic between high-bandwidth memory and the processor. Consequently, attention can become a major bottleneck during the prefill phase, particularly for long input sequences.

FlashAttention addresses this bottleneck through an input/output-aware implementation of exact attention. Instead of computing and storing the complete attention matrix, it divides the query, key and value tensors into smaller blocks that fit into fast on-chip memory. The attention result is then computed block by block using tiling and an incremental softmax calculation. This avoids repeatedly writing the large intermediate attention matrix to and reading it from GPU high-bandwidth memory. Importantly, FlashAttention changes the order in which attention is calculated but does not approximate the result; except for minor floating-point differences, it computes the same output as conventional attention [3].

This optimisation reduces the additional memory required for the attention computation from quadratic to linear in the sequence length, although the number of arithmetic operations remains quadratic. The original FlashAttention paper reported substantial wall-clock improvements, including a threefold speedup for GPT-2 with a sequence length of 1,024 tokens and a 2.4-fold speedup on long-range tasks with sequence lengths between 1,000 and 4,000 tokens. These results demonstrate that reducing memory transfers can improve execution time even without reducing the theoretical arithmetic complexity of attention [3].

3.2.2 KV Cache Optimisation

During autoregressive decoding, the keys and values computed for previous tokens are stored in the key-value (KV) cache. Reusing these tensors prevents the model from recomputing the attention representations of the complete preceding sequence at every decoding step. However, the cache grows linearly with the number of processed tokens, the number of Transformer layers, and the number of key-value heads. Its memory consumption can therefore become a major bottleneck for long contexts and large batches. Moreover, the cached tensors must be read repeatedly during decoding, so the KV cache also contributes to memory-bandwidth requirements.

Methods for reducing this bottleneck can be divided into three categories: architectural approaches that reduce the number of stored key-value heads, numerical approaches that store the cache at lower precision, and system-level approaches that manage cache allocation more efficiently.

KV-Cache Memory Management

Reducing the number or precision of cached tensors decreases the amount of information stored per token. A different problem arises from how the available cache memory is allocated. The length of a generated sequence is not normally known in advance, and the cache of each request grows dynamically. Allocating a large contiguous memory region for every request can therefore waste memory through reserved but unused capacity and through fragmentation.

PagedAttention addresses this allocation problem by dividing the KV cache into fixed-size blocks that need not occupy contiguous physical memory [11]. A block table maps the logical blocks belonging to each request to their physical locations, following a design analogous to virtual-memory paging. New blocks are allocated as the sequence grows, which limits unused reserved space and permits cache blocks to be shared when multiple generated sequences contain a common prefix.

PagedAttention is implemented in the vLLM serving system. Kwon et al. report that vLLM achieves near-zero waste in KV-cache memory and improves throughput by approximately $2\times$ to $4\times$ over the evaluated serving systems at a comparable latency level [11]. The throughput improvement is an indirect consequence of improved memory management: efficient allocation permits more requests to be included in a batch. PagedAttention does not inherently reduce the number of key and value elements associated with an individual token.

KV-Cache Quantisation

A second approach is to reduce the numerical precision of the cached key and value tensors. If a cache originally stored in 16-bit floating-point format is represented using 8-bit values, its raw storage requirement is approximately halved. More aggressive quantisation can produce greater reductions, but the keys and values do not necessarily exhibit the same numerical distributions and may therefore require different quantisation strategies.

KIVI is a tuning-free method that stores the KV cache using 2-bit quantisation [13]. Based on an analysis of the distributions of cached tensors, KIVI applies per-channel quantisation to the key cache and per-token quantisation to the value cache. The method also retains a small recent portion of the cache at higher precision before quantising it, thereby limiting the effect of outliers and recent-token quantisation errors.

Across the Llama, Falcon, and Mistral models evaluated by Liu et al., KIVI maintained approximately the same task performance while reducing peak memory consumption by up to $2.6\times$. The reduced memory requirement enabled batch sizes up to four times larger

and produced throughput improvements between $2.35\times$ and $3.47\times$ in the authors’ serving experiments [13]. These gains should not be interpreted as universal, however, because the realised acceleration depends on context length, batch size, quantisation overhead, and the availability of efficient low-precision cache kernels.

3.3 Decoding Acceleration and Request Scheduling

Autoregressive inference creates two distinct efficiency challenges. First, generating an individual response normally requires one sequential model invocation per output token. Second, serving several requests efficiently requires the available hardware to remain well utilised. Speculative decoding addresses the first challenge by accelerating individual sequences, whereas continuous batching addresses the second by dynamically scheduling concurrent requests.

3.3.1 Speculative Decoding

Standard autoregressive decoding generates one token per target-model forward pass, which limits generation speed because later tokens depend on earlier outputs. Speculative decoding uses a smaller draft model to propose K candidate tokens autoregressively [12]. The larger target model then evaluates these candidates together and accepts or rejects them using a modified rejection-sampling procedure. This permits several tokens to be produced during one target-model invocation while preserving the target model’s output distribution.

The achievable speedup depends on both the computational cost of the draft model and its acceptance rate. A draft model that frequently predicts the same tokens as the target allows more candidates to be accepted, whereas an inaccurate or computationally expensive draft model reduces the benefit. Leviathan et al. report a $2\times$ to $3\times$ acceleration for T5-XXL without changing the generated output distribution [12]. For edge deployment, this latency improvement must be balanced against the additional memory and computation required to store and execute the draft model.

3.3.2 Continuous Batching

Static batching processes a fixed group of requests together until every request in the batch has completed. Requests with different sequence lengths therefore cause inefficient utilisation, since completed requests leave unused batch positions while newly arriving requests must wait.

Continuous batching, also known as iteration-level scheduling, allows requests to be added to or removed from the active batch between decoding iterations [19]. This keeps the batch populated despite differences in request arrival times and output lengths, thereby improving hardware utilisation and aggregate throughput. ORCA demonstrated that iteration-level scheduling can substantially improve both throughput and latency compared with request-level scheduling for autoregressive Transformer serving [19].

Unlike speculative decoding, continuous batching does not reduce the number of decoding steps required by an individual sequence. It primarily benefits multi-user serving systems, whereas its usefulness is limited for single-user on-device inference where few independent requests are available for batching.

3.4 System-Level Inference Runtimes

The efficiency techniques discussed above require runtime support to produce practical improvements in memory consumption, latency, and throughput. The runtimes considered in this section were selected because they represent three different deployment objectives: llama.cpp focuses on portable local inference on commodity hardware, MLC-LLM uses machine-learning compilation to target heterogeneous platforms, and vLLM is primarily designed for high-throughput model serving.

3.4.1 llama.cpp

llama.cpp is a C/C++ runtime designed to execute large language models with minimal dependencies on a broad range of consumer hardware [8]. It provides optimised kernels for CPUs and several GPU backends, including Metal, CUDA, HIP, Vulkan, and SYCL. The runtime also supports low-bit weight quantisation and hybrid CPU–GPU execution, allowing models that exceed the available GPU memory to be distributed between system memory and the GPU.

Models are commonly stored in the GGUF format, which contains model weights and metadata and can represent weights quantised to different numerical precisions. GGUF is therefore a storage format rather than a quantisation algorithm. The portability and low setup requirements of llama.cpp make it well suited to local inference on laptops, desktop computers, and some mobile or embedded devices. Its performance, however, depends strongly on the quality of the kernels available for the target processor.

3.4.2 MLC-LLM

MLC-LLM is a deployment engine that uses machine-learning compilation to generate optimised implementations for different hardware backends [15]. A model is converted into an intermediate representation from which platform-specific code and kernels can be generated. This approach supports deployment across desktop GPUs, web browsers, iOS, and Android using backends such as CUDA, Metal, Vulkan, OpenCL, and WebGPU.

The principal strength of MLC-LLM is that the same deployment workflow can target heterogeneous devices while still applying hardware-specific optimisations. This is particularly relevant to edge deployment, where mobile and browser platforms expose different programming interfaces. The main trade-off is a more involved model-conversion and compilation workflow compared with directly loading a supported model in llama.cpp.

3.4.3 vLLM

vLLM is an inference and serving runtime designed primarily for efficient execution under concurrent workloads [11]. Its central contribution is PagedAttention, which manages the dynamically growing KV caches of requests in non-contiguous memory blocks. This reduces memory fragmentation and permits more requests to be processed concurrently. vLLM also combines this memory-management strategy with continuous batching and optimised execution kernels.

In the evaluation by Kwon et al., vLLM achieved between $2\times$ and $4\times$ higher throughput than the compared serving systems at a similar latency level [11]. Unlike llama.cpp and MLC-LLM, vLLM is principally intended for multi-user serving, where efficient batching and cache allocation are major concerns. It is therefore less directly suited to highly constrained single-user edge devices, although its techniques remain relevant to shared edge servers.

The three runtimes are not direct substitutes. llama.cpp prioritises simple and portable local execution, MLC-LLM prioritises deployment across heterogeneous platforms through compilation, and vLLM prioritises throughput and memory efficiency under concurrent serving workloads. Their suitability therefore depends more on the deployment environment than on a single performance ranking.

Runtime	Primary target	Main strength	Main limitation
llama.cpp	Local inference on CPUs and consumer GPUs	Portable C/C++ implementation, low-bit quantisation, and simple local deployment	Performance depends strongly on backend-specific kernel support
MLC-LLM	Mobile, browser, and heterogeneous devices	Platform-specific optimisation through machine-learning compilation	Requires model conversion and a more involved compilation workflow
vLLM	Concurrent model serving	High throughput through efficient KV-cache management and continuous batching	Primarily benefits multi-request serving rather than constrained single-user devices

Table 3.2: Comparison of the selected LLM inference runtimes.

3.5 Comparative Discussion and Edge-Deployment Guidelines

No single optimisation technique is superior across all deployment scenarios. The practical value of a method depends on the limiting resource of the target system, the characteristics of the workload, and the amount of quality degradation that can be accepted. For example, weight quantisation primarily reduces model memory and weight-transfer bandwidth, whereas FlashAttention mainly reduces intermediate memory traffic during attention computation. Speculative decoding targets the sequential nature of token generation, while continuous batching improves hardware utilisation when several requests are served concurrently. These techniques should therefore be regarded as complementary responses to different bottlenecks rather than as direct substitutes.

table 3.3 summarises the principal effects and trade-offs of the techniques discussed in this chapter. The entries describe their typical behaviour; the realised improvements depend on the model architecture, sequence length, batch size, hardware platform, numerical format, and availability of optimised runtime kernels.

Continuous batching is omitted from Table table 3.3 because it primarily improves aggregate throughput under concurrent serving workloads rather than the latency or memory requirements of single-user on-device inference. It nevertheless remains important for shared edge servers and data-centre deployments.

Technique	Primary bottleneck	Main benefit	Key trade-off
Weight quantisation	Weight memory and bandwidth	Reduces model size and weight-transfer cost	May reduce quality at low bit widths; speedup requires suitable low-precision kernels
Pruning	Model size and computation	Removes parameters and can reduce arithmetic cost	Structured pruning risks greater quality loss; unstructured pruning requires sparse hardware support
Knowledge distillation	Model memory and computation	Produces a smaller student model with lower inference cost	Requires additional training data, compute, and access to a capable teacher
FlashAttention	Attention memory traffic	Reduces intermediate storage and memory transfers	Does not reduce the quadratic arithmetic complexity of attention
KV-cache optimisation	Runtime memory and bandwidth	Supports longer contexts or more concurrent sequences	Benefits depend on the selected method, model architecture, and runtime support
Speculative decoding	Sequential token generation	Reduces target-model invocations per generated token	Requires an additional draft model and a sufficiently high acceptance rate

Table 3.3: Qualitative comparison of selected resource-efficient inference techniques.

3.5.1 The Efficiency–Accuracy Trade-off

Compression techniques reduce inference cost by discarding information or representing it less precisely. They therefore introduce a direct trade-off between resource consumption (memory) and model quality. The severity of this trade-off depends on several factors.

First, lower numerical precision generally increases quantisation error. Moving from FP16 to INT8 often preserves most of the original model quality, whereas more aggressive formats such as INT4 require more careful calibration and specialised quantisation methods. Below four bits per weight, degradation typically becomes more difficult to control. However, the relationship between bit width and quality is not uniform across models or tasks.

Second, some capabilities are more sensitive to compression than others. Tasks that depend on exact numerical manipulation, arithmetic reasoning, or small differences between output probabilities may degrade earlier than open-ended text-generation tasks. Evaluation should therefore include task-specific benchmarks rather than relying only on aggregate language-model perplexity.

Third, larger models often possess more redundancy and can tolerate stronger compression than smaller models. Consequently, a compressed large model may outperform a smaller uncompressed model while requiring a comparable amount of memory. The relevant deployment decision is therefore not simply whether to compress a model, but which combination of model scale and compression level provides the best quality within the available resource budget.

Not every efficiency technique introduces a quality trade-off. Implementation optimisations such as FlashAttention, PagedAttention, and continuous batching primarily change how computation or memory is organised. Similarly, correctly implemented speculative decoding preserves the target model’s output distribution. Their trade-offs instead concern implementation complexity, additional memory, workload requirements, or dependence on runtime and hardware support.

3.5.2 Memory as a Hard Edge-Deployment Constraint

Memory capacity is frequently the first hard constraint in edge deployment. A model cannot be executed locally unless its parameters, runtime state, and required intermediate data fit within the available memory. This distinguishes memory capacity from performance-related constraints such as compute throughput or memory bandwidth: insufficient compute may increase latency, but insufficient memory may prevent inference entirely.

The total inference-memory requirement consists of more than the model weights. It also includes intermediate activations, runtime buffers, and the KV cache. The weight memory is largely fixed for a selected model and numerical format, whereas the KV cache grows with sequence length, batch size, number of layers, and number of key–value heads. Long-context inference can therefore become memory-limited even when the model weights themselves fit on the device.

Weight quantisation is particularly important for edge deployment because it can reduce both the model footprint and the bandwidth required to load weights during decoding. Structured pruning and knowledge distillation can additionally reduce the number of operations, although they normally require more extensive model modification. KV-cache quantisation, Multi-Query Attention, and Grouped-Query Attention address the growing runtime memory associated with long contexts.

Once the model and its runtime state fit into memory, the dominant bottleneck may shift. During the prefill phase, long prompts can make attention computation and intermediate memory traffic important. During autoregressive decoding, repeatedly reading model weights and cached key–value tensors often makes memory bandwidth a major determinant of Time per Output Token (TPOT). Energy consumption also limits sustained performance on battery-powered devices, since high operating frequencies may cause rapid battery depletion or thermal throttling.

Memory should therefore be treated as an enabling constraint rather than the only measure of efficiency. Reducing the model sufficiently to fit on a device is the first requirement; achieving acceptable responsiveness subsequently depends on bandwidth, compute capability, energy consumption, and runtime implementation.

3.5.3 Technique Combination and Deployment Selection

The surveyed techniques can be combined because they operate at different levels of the inference stack. A quantised model may, for example, be executed using FlashAttention and a quantised KV cache, while speculative decoding is used to reduce the number of target-model decoding steps. In a concurrent serving environment, PagedAttention and continuous batching can additionally improve memory utilisation and aggregate throughput.

The benefits of combining techniques are not always additive. An unstructured pruning method may reduce the number of non-zero parameters without producing a latency improvement if the runtime lacks sparse kernels. Similarly, a low-precision format may reduce storage requirements but introduce conversion overheads when it is not natively supported by the processor. Speculative decoding may also be unattractive on devices with very limited memory because both the target and draft models must be stored and

executed.

For single-user inference on resource-constrained devices, the most practical starting point is usually a model whose weights have been quantised to a format efficiently supported by the target hardware. If long context lengths are required, this should be combined with architectural or numerical methods that reduce the KV cache. A runtime such as llama.cpp is suitable when portable local execution on CPUs or consumer GPUs is the main requirement, whereas MLC-LLM is more appropriate when deployment across mobile, browser, or heterogeneous accelerator backends is required.

For shared edge servers or data-centre deployment, the optimisation objective changes from fitting and accelerating one request to maximising throughput across many requests. In this setting, PagedAttention, continuous batching, and serving-oriented runtimes such as vLLM become more relevant. Speculative decoding may be beneficial in both settings, but its effectiveness depends on the cost and acceptance rate of the draft model.

The appropriate optimisation strategy should therefore be selected by first identifying the dominant deployment constraint. If the model does not fit in memory, compression is required. If decoding is bandwidth-bound, low-precision weights and KV-cache reduction are particularly relevant. If long-prompt processing is slow, attention implementations that reduce memory traffic can improve Time to First Token (TTFT). If the workload contains many concurrent requests, scheduling and cache-management techniques should be prioritised. This bottleneck-oriented approach avoids applying optimisations that reduce one resource while providing little practical benefit for the target workload.

In summary, resource-efficient inference is not achieved through a single universally optimal technique. Practical deployment requires a combination of model-level, algorithmic, and system-level optimisations selected according to the memory capacity, bandwidth, compute capability, energy budget, and workload characteristics of the target platform.

4 Conclusion

This paper presented the line of reasoning from the architecture of Transformer-based large language models to the techniques used to make their inference more resource-efficient. The Transformer relies on self-attention and feed-forward layers to generate contextualised token representations. During autoregressive inference, these operations are executed first for the complete input during the prefill phase and subsequently repeated for each generated token during the decode phase.

The analysis of this inference process showed that its resource requirements arise from several connected bottlenecks. Self-attention becomes increasingly expensive for long input sequences, the model parameters require substantial memory and must repeatedly be transferred during decoding, and the Key–Value cache grows with the number of processed tokens. As a result, inference efficiency depends not only on computational capacity, but also on memory capacity, memory bandwidth, energy consumption, and the characteristics of the target workload.

The surveyed optimisation methods address these bottlenecks at different levels. Model compression techniques, including quantisation, pruning, and knowledge distillation, reduce the memory and computational requirements of the model. Optimised attention computation and Key–Value cache management reduce memory traffic and intermediate storage. Speculative decoding and continuous batching improve token-generation latency or hardware utilisation, while runtimes such as `llama.cpp`, MLC-LLM, and vLLM combine algorithmic techniques with hardware-specific implementations.

The main takeaway is that resource-efficient inference is not achieved through one isolated optimisation. Practical deployment requires identifying the dominant bottleneck of the target system and combining compatible model-level, inference-level, decoding, and system-level techniques accordingly.

Bibliography

- [1] G. Bai, Z. Chai, C. Ling, S. Wang, J. Lu, C. Li, and L. Yao, “Beyond efficiency: A systematic survey of resource-efficient large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.00625>
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8 (): 8-bit matrix multiplication for transformers at scale,” *arXiv preprint arXiv:2208.07339*, 2022.
- [5] E. Frantar and D. Alistarh, “SPDY: Accurate pruning with speedup guarantees,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 6726–6743.
- [6] —, “Sparsegpt: Massive language models can be accurately pruned in one-shot,” *arXiv preprint arXiv:2301.00774*, 2023.
- [7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.17323>
- [8] G. Gerganov and contributors, “llama.cpp: C/C++ port of Meta’s LLaMA model,” GitHub repository, 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [9] Y. Gu, L. Dong, F. Wei, and M. Huang, “MiniLLM: Knowledge distillation of large language models,” in *The Twelfth International Conference on Learning Representations*, 2024.

- [10] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, “Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes,” in *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, 2023, pp. 8003–8017.
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, 2023, pp. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [12] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, 2023. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [13] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2bit quantization for kv cache,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, 2024. [Online]. Available: <https://arxiv.org/abs/2402.02750>
- [14] X. Ma, G. Fang, and X. Wang, “LLM-Pruner: On the structural pruning of large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: <https://arxiv.org/abs/2305.11627>
- [15] MLC-LLM Contributors, “MLC-LLM: Universal llm deployment engine with machine learning compilation,” <https://llm.mlc.ai/>, 2023, accessed: 2026-07-20.
- [16] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “LLaMA: Open and efficient foundation language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [18] Z. Wang, J. Wohlwend, and T. Lei, “Structured pruning of large language models,” *arXiv preprint arXiv:1910.04732*, 2019.
- [19] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation*,

ser. OSDI 22. USENIX Association, 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>

- [20] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, “A review on edge large language models: Design, execution, and applications,” *ACM Computing Surveys*, 2024. [Online]. Available: <https://dl.acm.org/doi/10.1145/3719664>
- [21] Y. Zhihang, S. Yuzhang, Z. Yang *et al.*, “Llm inference unveiled: Survey and roofline model insights,” 2024. [Online]. Available: <http://arxiv.org/abs/2402.16363>
- [22] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, and G. Dai, “A survey on efficient inference for large language models.” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14294>